

AWS

+

DevOps

+

Cloud Computing

+

Learning AWS and Terraform for Free: A Guide to Local Cloud Emulation Using Floci

5 min read · Jul 20, 2026



Rizal F

Follow



Listen



Share

Learning Cloud Infrastructure and Infrastructure as Code (IaC) often comes with a steep entry barrier: you either need a credit card to sign up for cloud services, or you live in constant fear of accidentally racking up a massive bill because you forgot to tear down a resource.

If you are running on a budget or working with limited hardware, there is a better

Open in app ↗

Sign up

Sign in

Medium

Search



lightweight cloud emulator) and deploy resources using **Terraform**.

Before we begin, ensure you have a running **Linux** environment with **Docker Engine** active to host the emulator, alongside the **Terraform CLI** and **AWS CLI** installed locally to manage and verify the infrastructure configuration.

The Stack: Why Floci?

While tools like LocalStack are popular, they can be heavy on system resources. If you are developing on standard or older hardware (like a reliable ThinkPad with dual-core processors and limited RAM), you need something fast and efficient.

Floci acts as a drop-in replacement for the AWS API. It runs inside a lightweight Docker container, listening on port 4566. When you run Terraform against it, Floci

mimics AWS's responses. It tracks states, creates mock endpoints, and lets you practice the exact same Infrastructure as Code (IaC) syntax you would use in production—minus the cloud bill and the high CPU usage.

1. Setting Up the Local Environment

First, ensure Docker is running on your machine. Launch the Floci emulator container with a single command:

```
# Install (macOS / Linux)
curl -fsSL https://floci.io/install.sh | sh

# Start the AWS emulator
floci start

# Check your environment
floci doctor
```

Floci is now active and ready to process AWS API calls locally on `localhost:4566`.

2. Writing the Terraform Configuration

Create a working directory and save the following code as `main.tf`.

The critical part here is the `endpoints` block within the AWS provider configuration. This block intercepts Terraform's default behavior (which is to call real AWS servers) and routes it to our local Docker container instead.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# CONFIG UTAMA BUAT FLOCI
provider "aws" {
  region                  = "us-east-1"
  access_key              = "mock_key_floci" # Bebas isi apa aja
  secret_key              = "mock_secret_floci"
  skip_credentials_validation = true
  skip_metadata_api_check  = true
```

```
skip_requesting_account_id = true

# Paksa Terraform nembak ke emulator lokal
endpoints {
  ec2 = "http://localhost:4566"
  s3  = "http://localhost:4566"
}

# --- RESOURCE TETEP SAMA ---

# 1. VPC
resource "aws_vpc" "local_vpc" {
  cidr_block = "10.0.0.0/16"
  tags = { Name = "Local-VPC" }
}

# 2. Subnet
resource "aws_subnet" "local_subnet" {
  vpc_id            = aws_vpc.local_vpc.id
  cidr_block        = "10.0.1.0/24"
  map_public_ip_on_launch = true
  tags = { Name = "Local-Subnet" }
}

# 3. EC2 Instance (Di-emulasi lokal oleh Floci via Docker)
resource "aws_instance" "local_web" {
  ami          = "ami-12345678" # Di Floci, ID AMI bisa di-mock bebas
  instance_type = "t2.micro"
  subnet_id    = aws_subnet.local_subnet.id

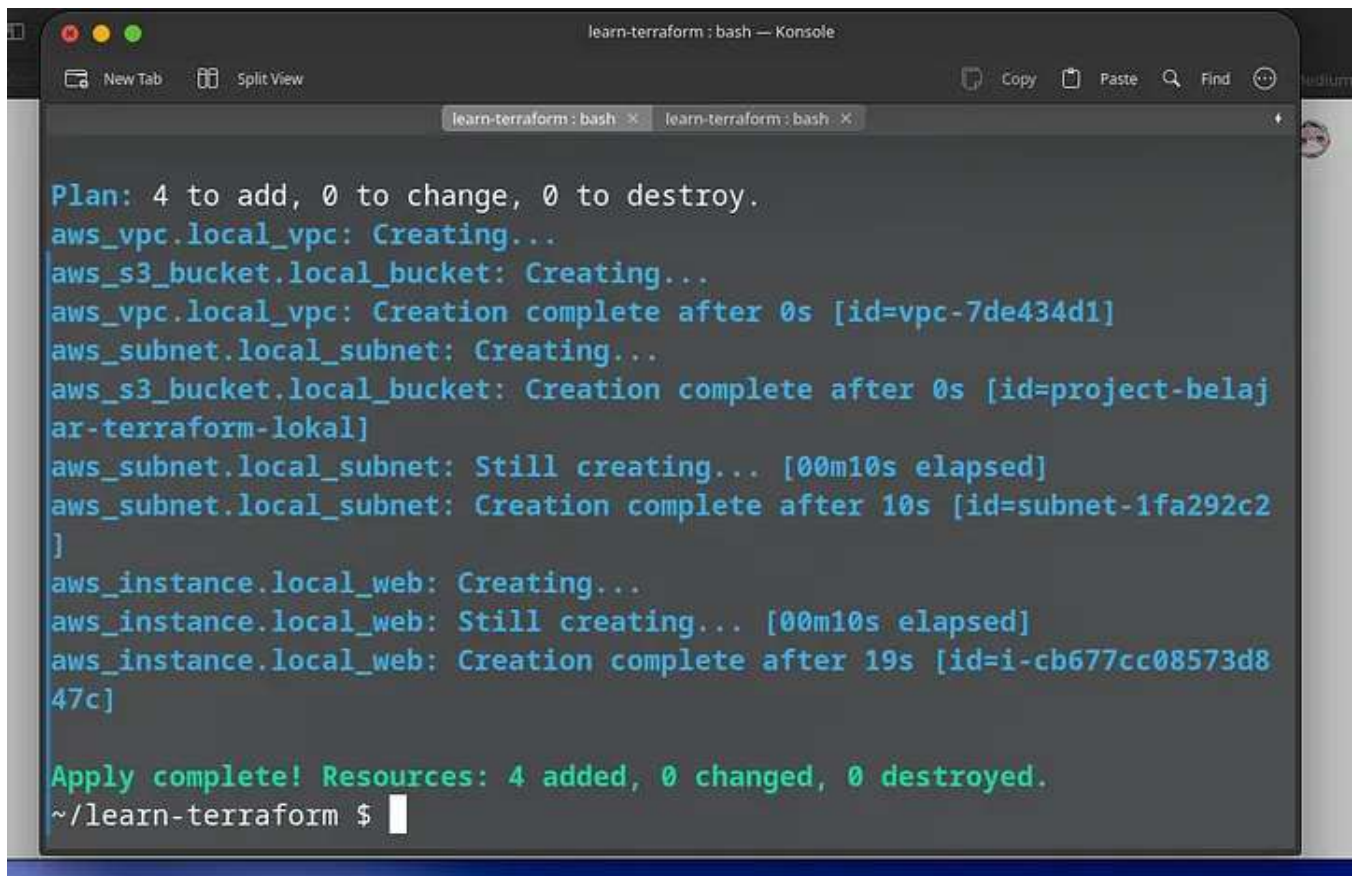
  tags = {
    Name = "Local-Nginx-Server"
  }
}

# 4. S3 Bucket
resource "aws_s3_bucket" "local_bucket" {
  bucket = "project-belajar-terraform-lokal"
  tags   = { Name = "LocalBucket" }
}
```

3. Deployment Workflow

```
terraform init
terraform plan
terraform apply --auto-approve
```

Because everything runs locally, the deployment finishes almost instantly. You should see an output indicating: Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

A screenshot of a terminal window titled 'learn-terraform : bash — Konsole'. The terminal shows the output of a Terraform apply command. It lists the creation of four resources: a VPC, an S3 bucket, a subnet, and an EC2 instance. The output indicates that all resources were created successfully. The final line shows 'Apply complete! Resources: 4 added, 0 changed, 0 destroyed.' followed by a prompt '~ /learn-terraform \$'.

```
Plan: 4 to add, 0 to change, 0 to destroy.
aws_vpc.local_vpc: Creating...
aws_s3_bucket.local_bucket: Creating...
aws_vpc.local_vpc: Creation complete after 0s [id=vpc-7de434d1]
aws_subnet.local_subnet: Creating...
aws_s3_bucket.local_bucket: Creation complete after 0s [id=project-belajar-terraform-lokal]
aws_subnet.local_subnet: Still creating... [00m10s elapsed]
aws_subnet.local_subnet: Creation complete after 10s [id=subnet-1fa292c2]
aws_instance.local_web: Creating...
aws_instance.local_web: Still creating... [00m10s elapsed]
aws_instance.local_web: Creation complete after 19s [id=i-cb677cc08573d847c]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
~/learn-terraform $
```

4. Verifying Resources via AWS CLI

To interact with these mock resources without logging into a web console, you can use the standard AWS CLI.

Get Rizal F's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

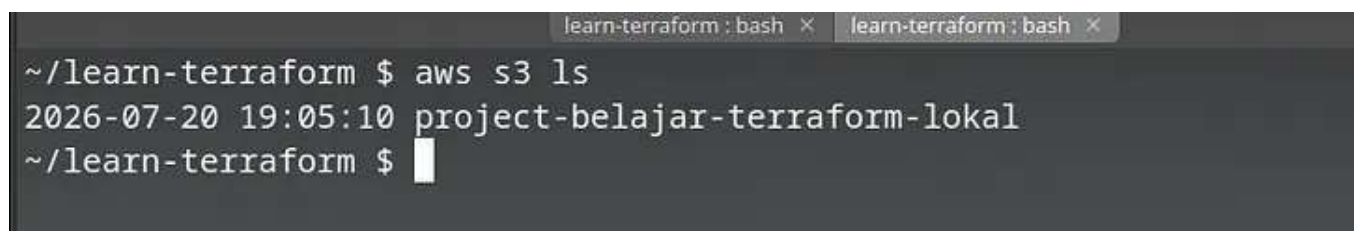
Floci provides a built-in environment utility to automatically configure your shell. Run this in your terminal to point your AWS CLI commands to the local container:

```
eval $(floci env)
```

Now, you can query your simulated cloud environment directly:

Check the S3 Bucket:

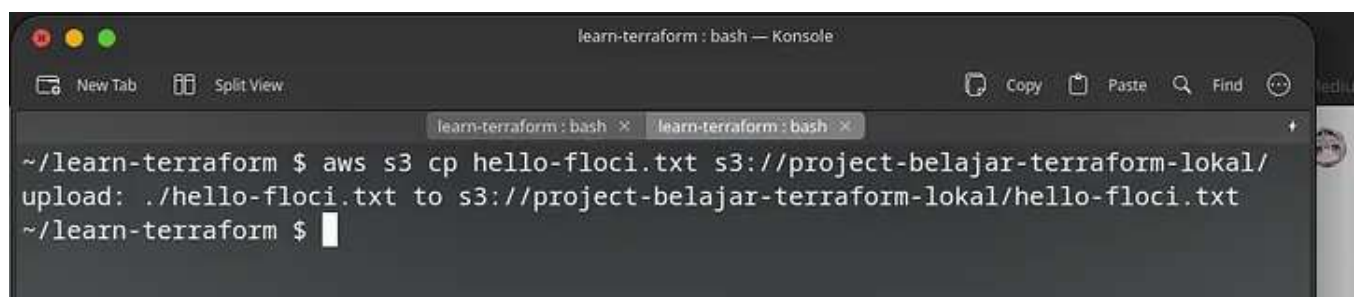
```
aws s3 ls
```

A terminal window with two tabs labeled 'learn-terraform : bash'. The prompt is '~ /learn-terraform \$'. The command 'aws s3 ls' is entered, and the output is '2026-07-20 19:05:10 project-belajar-terraform-lokal'. The prompt returns to '~ /learn-terraform \$' with a cursor.

```
~ /learn-terraform $ aws s3 ls
2026-07-20 19:05:10 project-belajar-terraform-lokal
~ /learn-terraform $
```

Upload a file to s3

```
echo "Hello World! Im Learning Floci" > hello-floci.txt
aws s3 cp hello-floci.txt s3://project-belajar-terraform-lokal/
```

A terminal window titled 'learn-terraform : bash — Konsole' with a menu bar (New Tab, Split View, Copy, Paste, Find, etc.). It has two tabs labeled 'learn-terraform : bash'. The prompt is '~ /learn-terraform \$'. The command 'aws s3 cp hello-floci.txt s3://project-belajar-terraform-lokal/' is entered, and the output is 'upload: ./hello-floci.txt to s3://project-belajar-terraform-lokal/hello-floci.txt'. The prompt returns to '~ /learn-terraform \$' with a cursor.

```
~ /learn-terraform $ aws s3 cp hello-floci.txt s3://project-belajar-terraform-lokal/
upload: ./hello-floci.txt to s3://project-belajar-terraform-lokal/hello-floci.txt
~ /learn-terraform $
```

Check The file

```
aws s3 ls s3://project-belajar-terraform-lokal
```

```
~/learn-terraform $ aws s3 ls s3://project-belajar-terraform-lokal
2026-07-20 20:32:20          31 hello-floci.txt
~/learn-terraform $
```

Download the File from s3

```
aws s3 cp s3://project-belajar-terraform-lokal/hello-floci.txt hello-back.txt
cat hello-back.txt
```

```
download: s3://project-belajar-terraform-lokal/hello-floci.txt to ./hello-back.txt
Hello World! Im Learning Floci
~/learn-terraform $
```

Remove the file in s3

```
~/learn-terraform $ aws s3 rm s3://project-belajar-terraform-lokal/hello-floci.txt
delete: s3://project-belajar-terraform-lokal/hello-floci.txt
~/learn-terraform $
```

Check the EC2 Instance Status:

```
aws ec2 describe-instances --query "Reservations[*].Instances[*].[InstanceId,St
```

```

learn-terraform : bash — Konsole
New Tab Split View Copy Paste Find
learn-terraform : bash x learn-terraform : bash x
~/learn-terraform $ aws s3 cp hello-floci.txt s3://project-belajar-terraform-lokal/
upload: ./hello-floci.txt to s3://project-belajar-terraform-lokal/hello-floci.txt
~/learn-terraform $ aws s3 ls s3://project-belajar-terraform-lokal
2026-07-20 20:32:20          31 hello-floci.txt
~/learn-terraform $ aws s3 cp s3://project-belajar-terraform-lokal/hello-floci.txt h
ello-back.txt
cat hello-back.txt
download: s3://project-belajar-terraform-lokal/hello-floci.txt to ./hello-back.txt
Hello World! Im Learning Floci
~/learn-terraform $ aws s3 rm s3://project-belajar-terraform-lokal/hello-floci.txt
delete: s3://project-belajar-terraform-lokal/hello-floci.txt
~/learn-terraform $ aws ec2 describe-instances --query "Reservations[*].Instances[*]
.[InstanceId,State.Name,Tags[0].Value]" --output table
-----
|                               DescribeInstances                               |
+-----+-----+-----+-----+
|  i-cb677cc08573d847c  |  running  |  Local-Nginx-Server  |
+-----+-----+-----+-----+
~/learn-terraform $

```

Check the Networks (VPCs):

```
aws ec2 describe-vpcs --query "Vpcs[*].[VpcId,CidrBlock]" --output table
```

```

~/learn-terraform $ aws ec2 describe-vpcs --query "Vpcs[*].[VpcId,CidrBlock]" --outp
ut table
-----
|          DescribeVpcs          |
+-----+-----+-----+-----+
|  vpc-default  |  172.31.0.0/16  |
|  vpc-7de434d1 |  10.0.0.0/16   |
+-----+-----+-----+-----+
~/learn-terraform $

```

Under the Hood: Emulator vs. Virtualization

One important detail to keep in mind when using this setup: **Floci is an API emulator, not a hypervisor.**

When you deploy an `aws_instance`, the emulator registers the API call, creates the metadata, and reports the instance as `running`. However, it does not spin up an actual Ubuntu virtual machine inside your computer. You won't be able to SSH into port 22 or curl port 80 of that specific IP address.

This behavior is intentional. By focusing strictly on API validation rather than heavy OS virtualization, the setup keeps your laptop running cool and smooth while allowing you to master Terraform syntax, state management, and the AWS CLI toolchain.

Conclusion

You don't need expensive subscriptions or cloud credits to start learning DevOps and Cloud Engineering. By combining Terraform with local emulators like Floci, you can build, break, and rebuild complex cloud architectures right from your local terminal.

The complete repository, including files and setup scripts, is available on [Github](#). Feel free to clone it and start testing your own configurations.

AWS

+

DevOps

+

Cloud Computing

+

[Follow](#)

Written by Rizal F

4 followers · 5 following